

Hash-Based MAC

Hash-based MAC is often faster than block cipher-based MACs.

How? Heuristic constructions:

- **Prepend Key:** $MAC_k^{PK}(m) = h(k||m)$
- **Append Key:** $MAC_k^{AK}(m) = h(m||k)$
- **Message-in-the-Middle:** $MAC_k^{MitM}(m) = h(k||m||k)$

Are these secure assuming CRHF? OWF? Both? **No, since it's a MAC**

But: all are "secure in the random oracle model:" when the hash function is assumed to behave like a random function.

Hash-based MAC: HMAC

HMAC uses an unkeyed hash function h :

$$HMAC_k(x) = h(k \oplus opad || h(k \oplus ipad || x))$$

opad, *ipad*: fixed sequences (of 36x, 5Cx respectively).

It is a secure MAC under "reasonable assumptions"

Widely deployed

More results, more exposure $\rightarrow$ confidence!

Hash functions are useful for MACs in another way:

Hash the MAC for efficiency.

Accumulators

Generalization of collision-resistant hash

- Input is a **sequence (ordered list)** of messages
- Output is n -bit **digest**, denoted Δ

Collision resistance accumulator means that it is hard to find two different message lists that have the same digest.

IRL, you don't have the full list in advance. You are computing the digest online (on the fly): *accumulating* new messages to digest.

Accumulator Components

Digest function $\Delta : \{m_i \in \{0, 1\}^*\} \rightarrow \{0, 1\}^n$

- Also called accumulate function
- Collision-resistance requirement

Proof of inclusion: *PoI* and *VerPoI*

- *PoI* function: compute Proof of Inclusion
- *VerPoI* function: verify Proof of Inclusion
- Optional, also Proof of Non-Inclusion (*PoNI*)

Extending the Sequence: Extend function with optional *PoC* and *VerPoC*

- *PoC*: Proof of Consistency (from old digest to new)
- *VerPoC* function: verify PoC

Correctness and Security for PoI and PoC

If you output in an honest way, then the validation algorithm must accept that proof.

Correctness means that on input a valid PoI, VerPoI will output 1 (same for PoC).

For PoI: security means that a PPT adversary cannot forge a valid PoI for a message that is not in the hashed list.

For PoC: security means that a PPT adversary cannot forge a valid PoC for an invalid digest extension.

We will study three accumulator types

- Merkle-Damgard Accumulators
- Merkle Trees
- Blockchains

The Merkle-Damgard Accumulator

Idea: hash iteratively, message by message:

$$\Delta(m_1, \dots, m_l) = h(\Delta(m_1, \dots, m_{l-1}) || 1 || m_l) ; \Delta(m_1) = h(0^{n+1} || m_1)$$

If h is a CRHF, then Δ is a collision-resistant digest.

Merkle-Damgard Length-Padding

Aka Merkle-Damgard Strengthening

Let $pad(x) = 1 || 0^k || bin(|x|)$; $x' = x || pad(x)$ (where $bin(|x|)$ is the n -bit binary representation of $|x|$)

For $i = 1, \dots, l$ where $l = \frac{|x'|}{n}$, and let x'_i = the i^{th} n -bit block of x'

Apply the construction in the prior section to obtain the digest of x'

This is just a high level idea, care is needed to avoid collisions.

The Digest-Chain Extend Function

Beyond digest and collision resistance: sequence-related integrity mechanisms

For digest-chain, the **extend function**

Not secure to be used to construct a MAC

The Merkle Tree Construction

Reduce length of "proofs" - send less hashes of "other messages"

Use binary tree.

Hash leaves, hash those hashes together, hash the hashes of hashes, etc..

$$MT\Delta(M) = \begin{cases} h(m_1) & \text{If } L = 0 \\ h(MT\Delta(m_1, \dots, m_{2^{L-1}}) || MT\Delta(m_{2^{L-1}+1}, \dots, m_{2^L})) & \text{otherwise} \end{cases}$$

$m_1, \dots, m_{32}$; $|PoI| = 5 = \log_2(32)$; Number of Levels = 6 (5+Root Hash)

Two-Layered Merkle Tree

Short digest validates integrity of large object (often, object consists of multiple "files")

Merkle Tree; integrity for many messages

- Hash each "message" in the block separately, then hash of hashes.

$$\delta = h(h(m_1)||h(m_2)||h(m_3)||h(m_4))$$

- Validate each "message" independently

You are given the root of the MT

PoI: message must be part of root

How to compute root? PoI must be the full list?

Verify: $\delta' = h(h(m_1)||...||h(m_4))$, $\delta \stackrel{?}{=} \delta'$

Can verify with just m_1 and the other hashes ($h(m_2)||h(m_3)||h(m_4)$) (hash m_1 , concatenate and verify)

Therefore, to verify you must send the message that is supposedly in the tree with the PoI being the hash of all the other messages

Differences between Accumulators and Hash Function

- Hash takes one messages and outputs one digest
- Accumulators take a set of messages and can extend the digest with a new set